

CITED REFERENCES AND FURTHER READING:

- Brigham, E.O. 1974, *The Fast Fourier Transform* (Englewood Cliffs, NJ: Prentice-Hall).[1]
- Nussbaumer, H.J. 1982, *Fast Fourier Transform and Convolution Algorithms* (New York: Springer).
- Elliott, D.F., and Rao, K.R. 1982, *Fast Transforms: Algorithms, Analyses, Applications* (New York: Academic Press).
- Walker, J.S. 1996, *Fast Fourier Transforms*, 2nd ed. (Boca Raton, FL: CRC Press)
- Bloomfield, P. 1976, *Fourier Analysis of Time Series – An Introduction* (New York: Wiley).
- Van Loan, C. 1992, *Computational Frameworks for the Fast Fourier Transform* (Philadelphia: S.I.A.M.).[2]
- Press, W.H., Teukolsky, S.A., Vetterling, W.T., and Flannery, B.P. 1996, *Numerical Recipes in Fortran 90: The Art of Parallel Scientific Computing* (Cambridge, UK: Cambridge University Press), §22.4.[3]
- Yip, P.C.Y. 1976, “Some Aspects of the Zoom Transform,” *IEEE Transactions on Computers*, vol. C-25, pp. 287–296.[4]
- Hung, E.K.L. 1981, “A Multiresolution Sampled-Data Spectrum Analyzer for a Detection System,” *IEEE Transactions on Acoustics, Speech and Signal Processing*, vol. ASSP-29, pp. 163–170.[5]
- de Wild, R., Nieuwkerk, L.R., and van Sinttruyen, J.S. 1987, “Method for Partial Spectrum Computation,” *IEE Proceedings F (Radar and Signal Processing)*, vol. 134, pp. 659–666[6]
- Beauchamp, K.G. 1984, *Applications of Walsh Functions and Related Functions* (New York: Academic Press) [non-Fourier transforms].
- Pollard, J.M. 1971, “The Fast Fourier Transform in a Finite Field,” *Mathematics of Computation*, vol. 25, pp. 365–374.[7]
- McClellan, J.H., and Rader, C.M. 1979, *Number Theory in Digital Signal Processing* (New York: Prentice-Hall).[8]
- Heideman, M.T., Johnson, D.H., and Burris, C.S. 1984, “Gauss and the History of the Fast Fourier Transform,” *IEEE ASSP Magazine*, pp. 14–21 (October).

12.3 FFT of Real Functions

It happens frequently that the data whose FFT is desired consist of real-valued samples f_j , $j = 0 \dots N - 1$. To use `four1`, we put these into a complex array with all imaginary parts set to zero. The resulting transform F_n , $n = 0 \dots N - 1$ satisfies $(F_{N-n})^* = F_n$. Since this complex-valued array has real values for F_0 and $F_{N/2}$, and $(N/2) - 1$ other independent values $F_1 \dots F_{N/2-1}$, it has the same $2(N/2-1)+2 = N$ “degrees of freedom” as the original, real data set. However, the use of the full complex FFT algorithm for real data is inefficient, both in execution time and in storage required. You would think that there is a better way.

Actually, there are *two* better ways.

12.3.1 Transform of Two Real Functions Simultaneously

The first better way is “mass production”: Pack two separate real functions into the input array in such a way that their individual transforms can be separated from the result. This may remind you of a one-cent sale, at which you are coerced to purchase two of an item when you only need one. However, remember that for correlations and convolutions the Fourier transforms of two functions are involved, and this is a handy way to do them both at once.

Here is how to exploit the symmetry of the FFT to handle two real functions at once: Pack the two data arrays as the real and imaginary parts, respectively, of the complex input array of `four1` and take the transform. This gives

$$H_n = \sum_j e^{2\pi i j n / N} (f_j + i g_j) \quad (12.3.1)$$

Now look at the $N - n$ component, and take its complex conjugate,

$$(H_{N-n})^* = \left(\sum_j e^{2\pi i j (N-n) / N} (f_j + i g_j) \right)^* = \sum_j e^{2\pi i j n / N} (f_j - i g_j) \quad (12.3.2)$$

where we have used $f_j^* = f_j$ and $g_j^* = g_j$. Now, adding and subtracting equations (12.3.1) and (12.3.2) gives

$$H_n + H_{N-n}^* = 2F_n, \quad H_n - H_{N-n}^* = 2iG_n \quad (12.3.3)$$

Equations (12.3.3) with $n = 0, 1, \dots, N/2$ easily yield the independent (zero and positive frequency) components of the two desired transforms F_n and G_n . Note that $F_0, G_0, F_{N/2}$, and $G_{N/2}$ are real (using $H_0 = H_N$), but that the other values are, in general, complex.

What about the reverse process? This is even easier. Using the symmetries $F_{N-n} = F_n^*$ and $G_{N-n} = G_n^*$, form $F_n + iG_n$ for $0 \leq n < N$. Now take the inverse FFT. The real and imaginary parts of the resulting complex array are the two desired real functions.

The only potential drawback of this method occurs if f and g are very different in scale. Then roundoff error can cause the smaller function's FFT to be inaccurate.

12.3.2 FFT of a Single Real Function

To implement the second method, which allows us to perform the FFT of a *single* real function without redundancy, we split the data set in half, thereby forming two real arrays of half the size. We can apply the method above to these two, but of course the result will not be the transform of the original data. It will be a schizophrenic combination of two transforms, each of which has half of the information we need. Fortunately, this schizophrenia is treatable. It works like this:

The right way to split the original data is to take the even-numbered f_j as one data set, and the odd-numbered f_j as the other. The beauty of this is that we can take the original real array and treat it as a complex array h_j of half the length. The first data set is the real part of this array, and the second is the imaginary part, just as was described above. No repacking is required. In other words, $h_j = f_{2j} + i f_{2j+1}$, $j = 0, \dots, N/2 - 1$. We submit this to `four1`, and it gives back a complex array $H_n = F_n^e + i F_n^o$, $n = 0, \dots, N/2 - 1$ with

$$\begin{aligned} F_n^e &= \sum_{k=0}^{N/2-1} f_{2k} e^{2\pi i k n / (N/2)} \\ F_n^o &= \sum_{k=0}^{N/2-1} f_{2k+1} e^{2\pi i k n / (N/2)} \end{aligned} \quad (12.3.4)$$

The previous discussion tells us how to separate the two transforms F_n^e and F_n^o out of H_n . How do you work them into the transform F_n of the original data set f_j ? Simply glance back at equation (12.2.3):

$$F_n = F_n^e + e^{2\pi i n/N} F_n^o \quad n = 0, \dots, N-1 \quad (12.3.5)$$

Expressed directly in terms of the transform H_n of our real (masquerading as complex) data set, the result is

$$F_n = \frac{1}{2}(H_n + H_{N/2-n}^*) - \frac{i}{2}(H_n - H_{N/2-n}^*)e^{2\pi i n/N} \quad n = 0, \dots, N-1 \quad (12.3.6)$$

A few remarks:

- Since $F_{N-n}^* = F_n$, there is no point in saving the entire spectrum. The positive frequency half is sufficient and can be stored in the same array as the original data. The operation can, in fact, be done in place.
- Even so, we need values H_n , $n = 0, \dots, N/2$, whereas `four1` gives only the values $n = 0, \dots, N/2 - 1$. Symmetry to the rescue, $H_{N/2} = H_0$.
- The values F_0 and $F_{N/2}$ are real and independent. In order to actually get the entire F_n in the original array space, it is convenient to put $F_{N/2}$ into the imaginary part of F_0 .
- Despite its complicated form, the process above is invertible. First peel $F_{N/2}$ out of F_0 . Then construct

$$\begin{aligned} F_n^e &= \frac{1}{2}(F_n + F_{N/2-n}^*) \\ F_n^o &= \frac{1}{2}e^{-2\pi i n/N}(F_n - F_{N/2-n}^*) \quad n = 0, \dots, N/2 - 1 \end{aligned} \quad (12.3.7)$$

and use `four1` to find the inverse transform of $H_n = F_n^{(1)} + iF_n^{(2)}$. Surprisingly, the actual algebraic steps are virtually identical to those of the forward transform.

Here is a representation of what we have said:

```
void realft(VecDoub_IO &data, const Int isign) {
```

`fourier.h`

Calculates the Fourier transform of a set of n real-valued data points. Replaces these data (which are stored in array `data[0..n-1]`) by the positive frequency half of their complex Fourier transform. The real-valued first and last components of the complex transform are returned as elements `data[0]` and `data[1]`, respectively. n must be a power of 2. This routine also calculates the inverse transform of a complex data array if it is the transform of real data. (Result in this case must be multiplied by $2/n$.)

```
    Int i,i1,i2,i3,i4,n=data.size();
    Doub c1=0.5,c2,h1r,h1i,h2r,h2i,wr,wi,wpr,wpi,wtemp;
    Doub theta=3.141592653589793238/Doub(n>>1);    Initialize the recurrence.
    if (isign == 1) {
        c2 = -0.5;
        four1(data,1);
    } else {
        c2=0.5;
        theta = -theta;
    }
    wtemp=sin(0.5*theta);
    wpr = -2.0*wtemp*wtemp;
    wpi=sin(theta);
    wr=1.0+wpr;
    wi=wpi;
```

The forward transform is here.

Otherwise set up for an inverse transform.